



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Course: BVWS103-OWASP Top 10 Vulnerabilities And
Exploitation Techniques

Student Name: Abubakari-Sadique Hamidu
Student ID: 2025/FWSD/11392

Instructor: Aminu Iddris (CCNA, CompTIA, CEH, OSCP, CISSP, CISM)

Lab13: Session Hijack

Lab 13: Session Hijacking and Mitigation

Objective

The purpose of this lab was to:

1. Understand how session hijacking works.
2. Simulate a session hijacking attack.
3. Implement mitigation techniques to prevent session hijacking.

Task 1: Simulating Session Hijacking

1. Setup of Vulnerable Application

A simple PHP login system was created using login.php and profile.php.
The application used PHP sessions (PHPSESSID) to maintain user authentication.

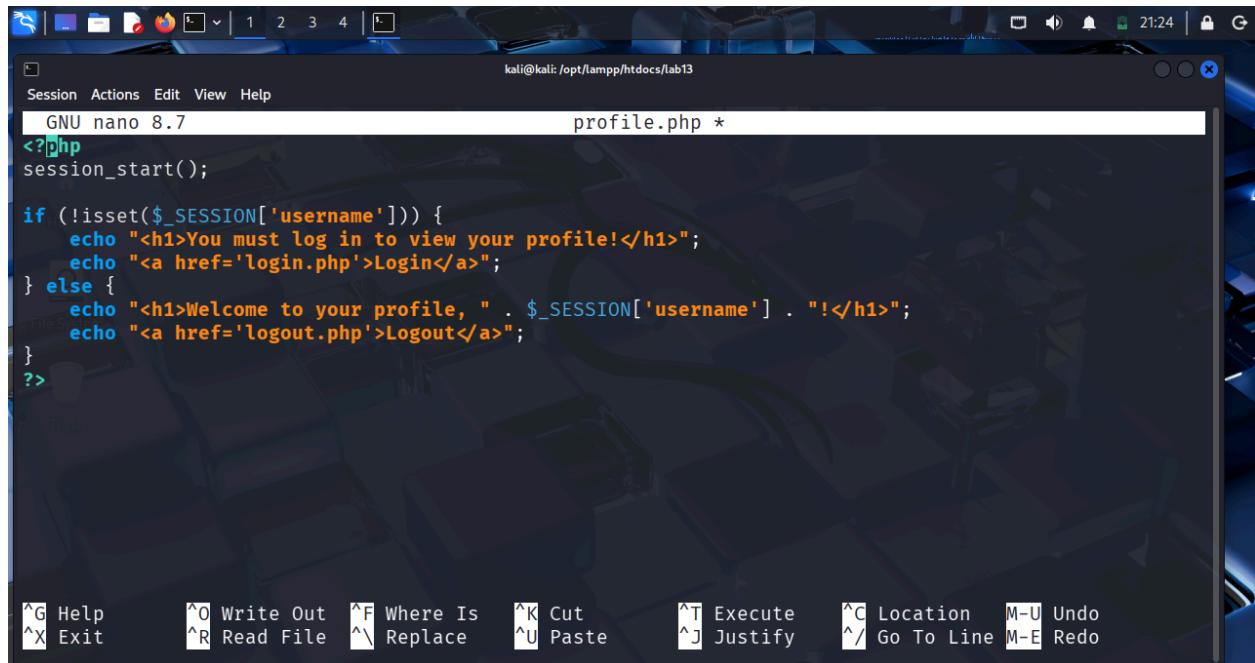
```
kali@kali: /opt/lampp/htdocs
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~$
$ cd /opt/lampp/htdocs
(kali@kali)-[/opt/lampp/htdocs]
$ mkdir lab13
(kali@kali)-[/opt/lampp/htdocs]
$ chmod 777 lab13
(kali@kali)-[/opt/lampp/htdocs]
$
```

```
kali@kali: /opt/lampp/htdocs/lab13
Session Actions Edit View Help
GNU nano 8.7 login.php *
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $username = $_POST['username'];
    $password = $_POST['password'];

    // Check if the username and password match
    if (isset($users[$username]) && $users[$username] == $password) {
        $_SESSION['username'] = $username;
        echo "<h1>Welcome, " . $username . "!</h1>";
        echo "<a href='profile.php'>Go to Profile</a>";
    } else {
        echo "<h1>Invalid credentials!</h1>";
    }
}
?>

<form method="POST">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required><br>
    <label for="password">Password:</label>

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_/ Go To Line  M-E Redo
```



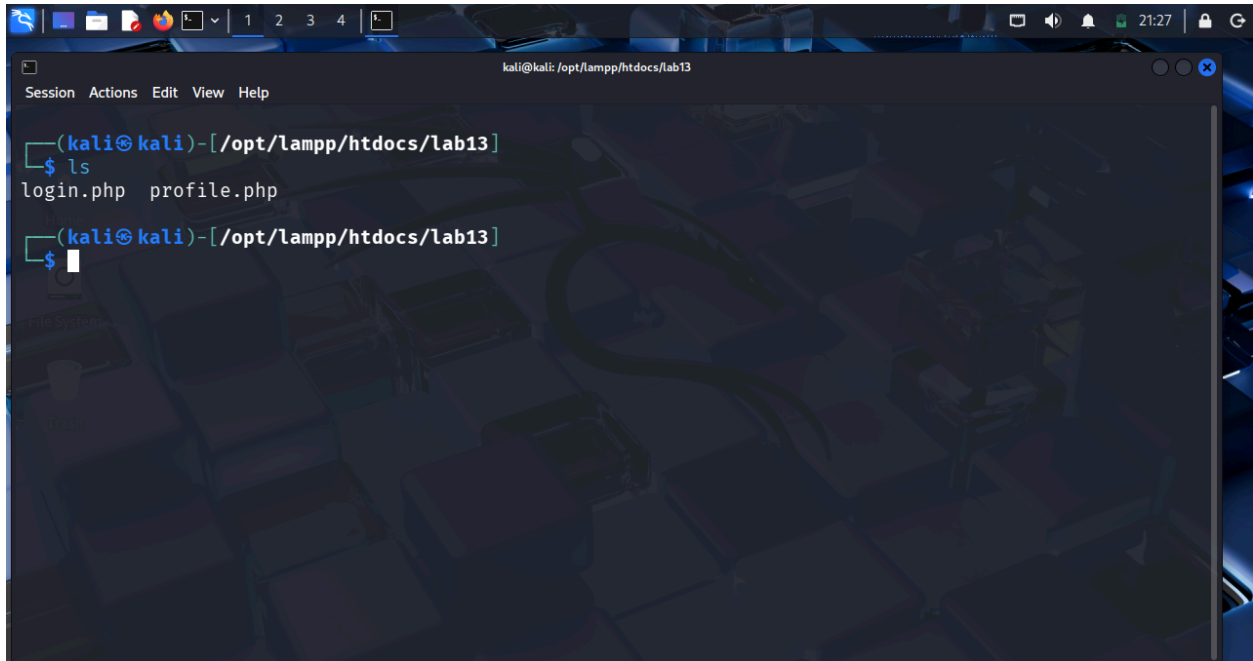
The image shows a terminal window on a Kali Linux system. The nano text editor is open, editing a file named `profile.php`. The code in the editor is as follows:

```
GNU nano 8.7 profile.php *
<?php
session_start();

if (!isset($_SESSION['username'])) {
    echo "<h1>You must log in to view your profile!</h1>";
    echo "<a href='login.php'>Login</a>";
} else {
    echo "<h1>Welcome to your profile, " . $_SESSION['username'] . "!</h1>";
    echo "<a href='logout.php'>Logout</a>";
}
?>
```

The terminal window includes a menu bar with options: Session, Actions, Edit, View, Help. At the bottom, there is a keyboard shortcuts menu:

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify	^/_ Go To Line	M-E Redo



A terminal window titled 'kali@kali: /opt/lampp/htdocs/lab13' with a menu bar (Session, Actions, Edit, View, Help). The prompt is '(kali@kali)-[/opt/lampp/htdocs/lab13]'. The user enters '\$ ls', and the output is 'login.php profile.php'. The prompt returns to '(kali@kali)-[/opt/lampp/htdocs/lab13]' with a new line ready for input.

```
(kali@kali)-[/opt/lampp/htdocs/lab13]
$ ls
login.php  profile.php
(kali@kali)-[/opt/lampp/htdocs/lab13]
$
```



A terminal window titled 'root@kali: /opt/lampp/htdocs/lab13' with a menu bar (Session, Actions, Edit, View, Help). The prompt is '(kali@kali)-[/opt/lampp/htdocs/lab13]'. The user enters '\$ ls', and the output is 'login.php profile.php'. The prompt returns to '(kali@kali)-[/opt/lampp/htdocs/lab13]'. The user enters '\$ sudo su', and the output is '[sudo] password for kali:'. The prompt changes to '(root@kali)-[/opt/lampp/htdocs/lab13]'. The user enters '# /opt/lampp/lampp start', and the output is 'Starting XAMPP for Linux 8.2.12-0 ... XAMPP: Starting Apache ... ok. XAMPP: Starting MySQL ... ok. XAMPP: Starting ProFTPD ... ok.'. The prompt returns to '(root@kali)-[/opt/lampp/htdocs/lab13]' with a new line ready for input.

```
(kali@kali)-[/opt/lampp/htdocs/lab13]
$ ls
login.php  profile.php
(kali@kali)-[/opt/lampp/htdocs/lab13]
$ sudo su
[sudo] password for kali:
(root@kali)-[/opt/lampp/htdocs/lab13]
# /opt/lampp/lampp start
Starting XAMPP for Linux 8.2.12-0 ...
XAMPP: Starting Apache ... ok.
XAMPP: Starting MySQL ... ok.
XAMPP: Starting ProFTPD ... ok.
(root@kali)-[/opt/lampp/htdocs/lab13]
$
```

2. Successful Login

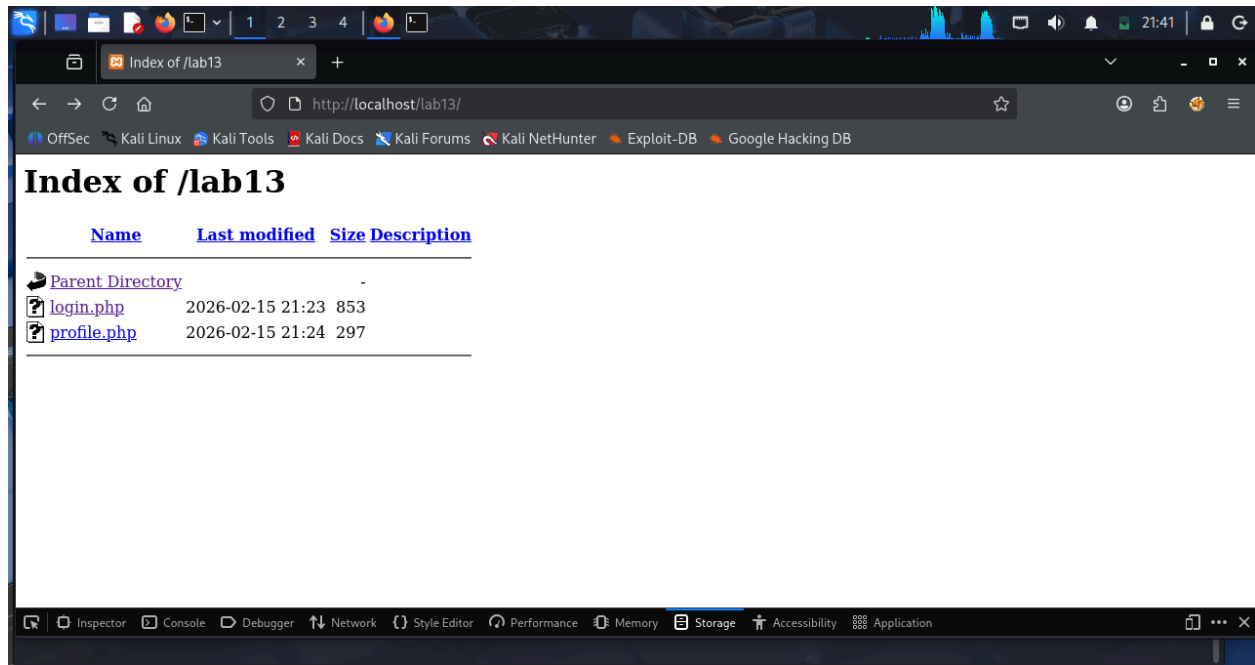
The application was accessed via:

<http://localhost/Lab13/login.php>

Login credentials used:

- Username: admin
- Password: password123

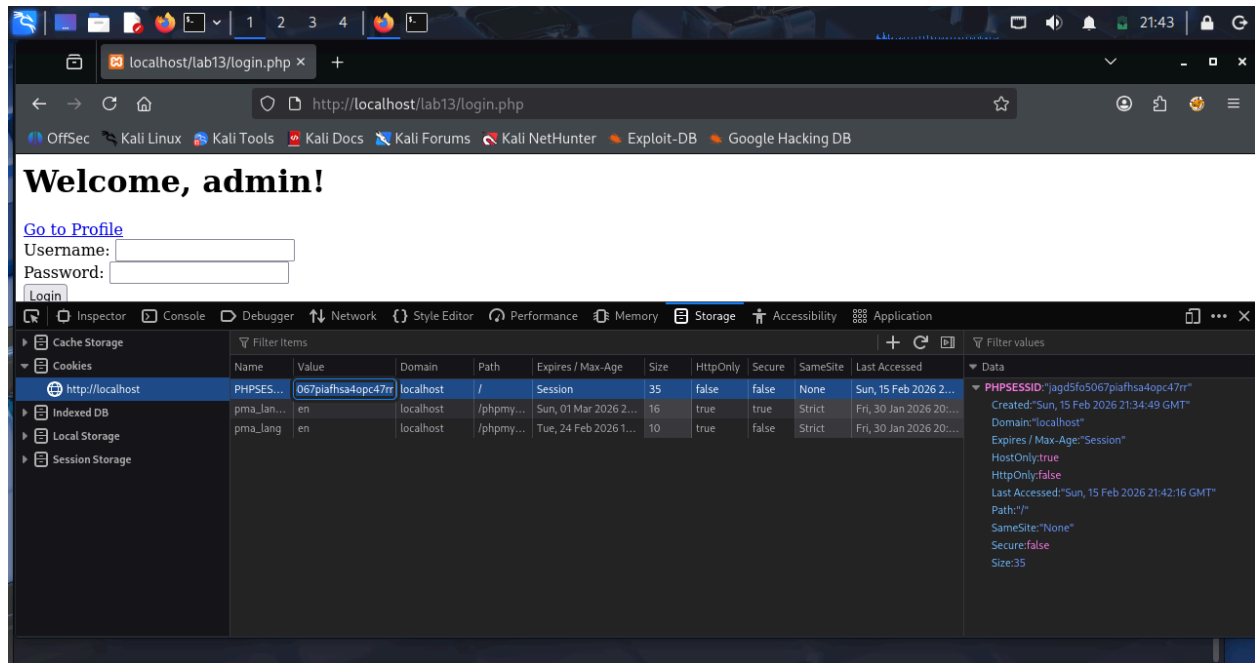
After successful login, the system created a session and granted access to the profile page.



3. Extracting the Session ID

Using browser Developer Tools:

- Navigated to **Application** → **Cookies** → **localhost**
- Located the session cookie named PHPSESSID
- Copied the session ID value



4. Simulating the Attack

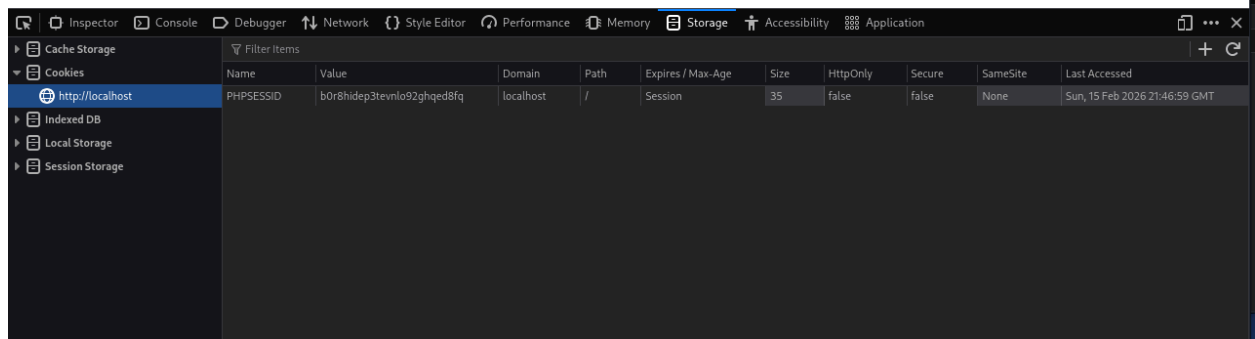
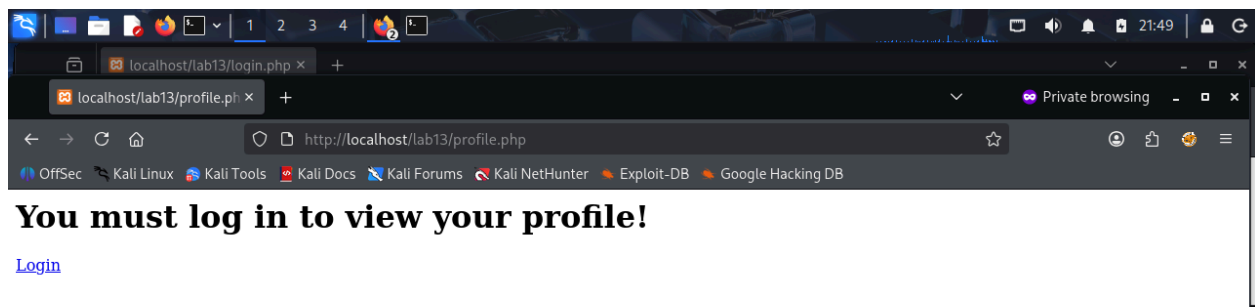
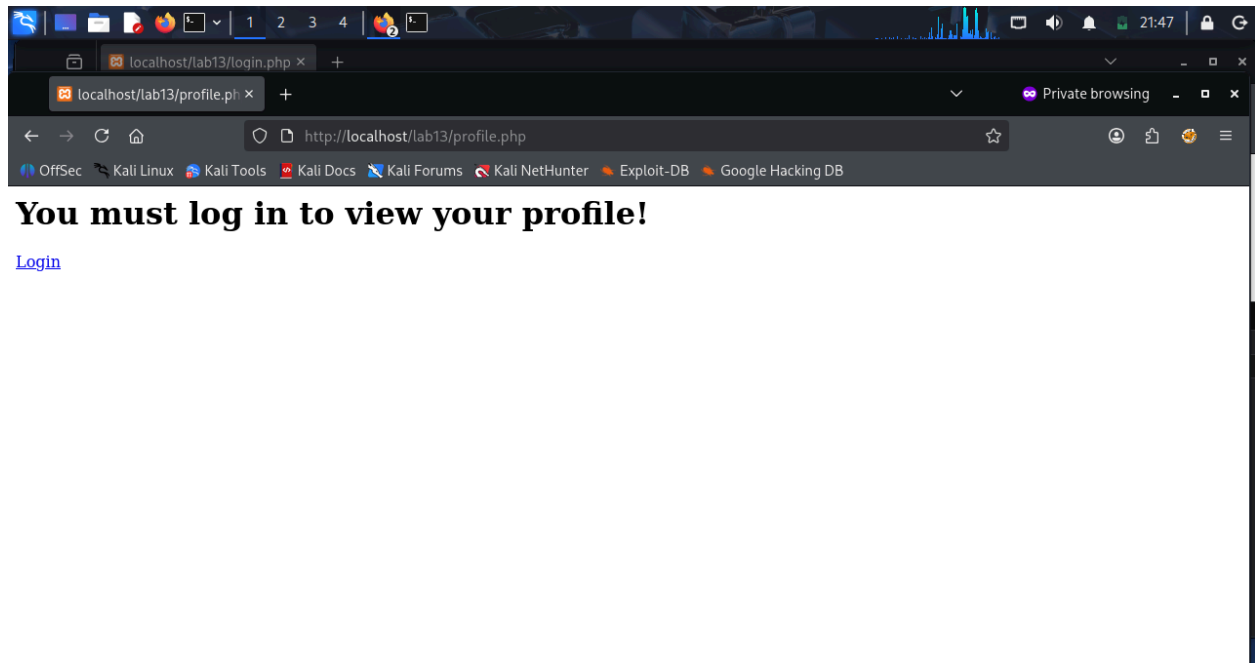
An Incognito window was opened.

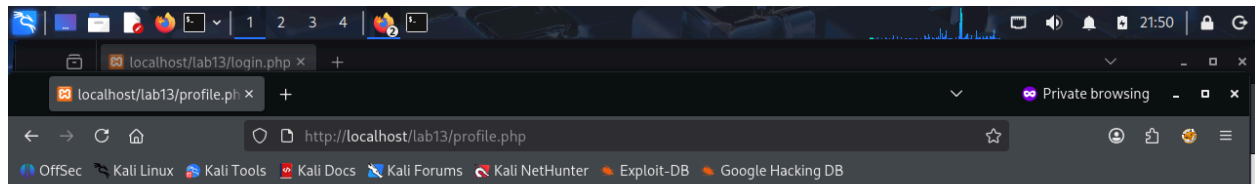
Steps performed:

1. Navigated to profile.php
2. Manually inserted the stolen PHPSESSID into cookies
3. Refreshed the page

Result:

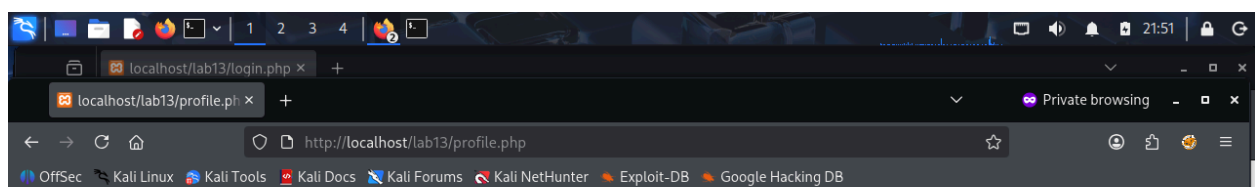
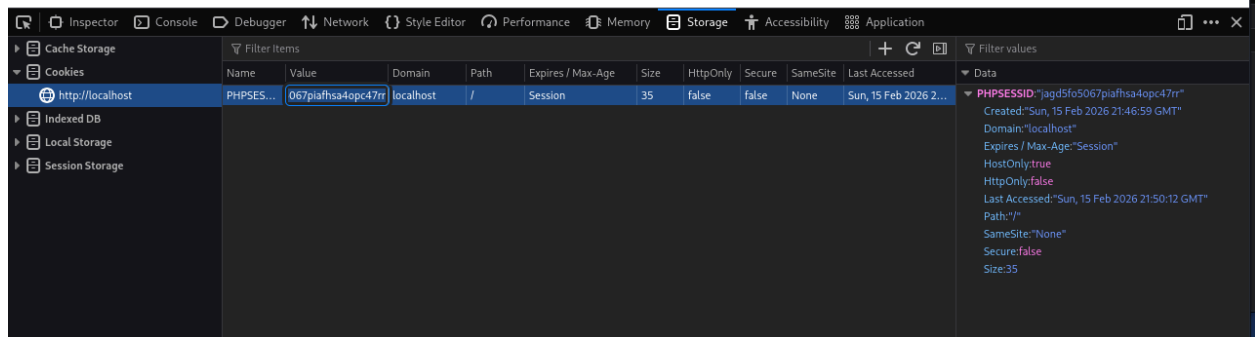
The profile page was accessed without logging in, proving that session hijacking was successful.





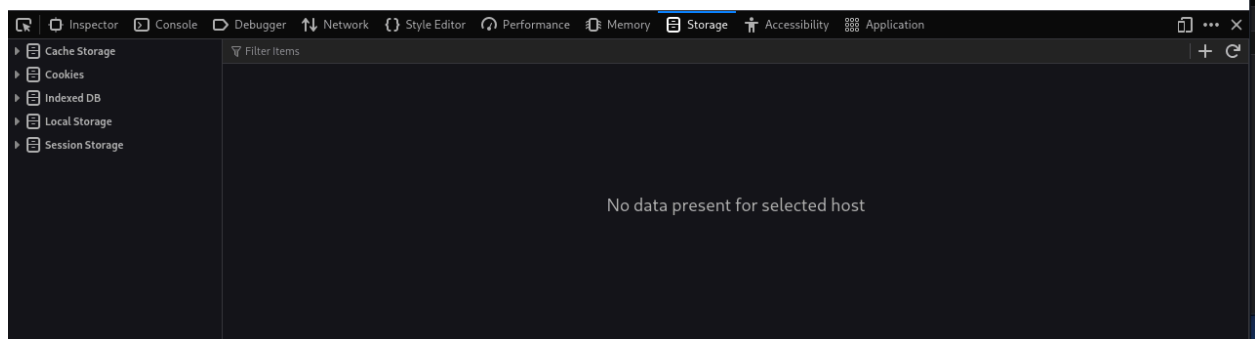
You must log in to view your profile!

[Login](#)



Welcome to your profile, admin!

[Logout](#)



Explanation of the Vulnerability

Session hijacking works by stealing a valid session ID from a legitimate user.

Since the server uses the session ID to identify authenticated users, an attacker who obtains it can impersonate the victim without knowing their password.

The vulnerability existed because:

- The session ID was not regenerated after login.
- No session timeout was implemented.
- Cookie security flags were not configured.

Task 2: Mitigation Measures Implemented

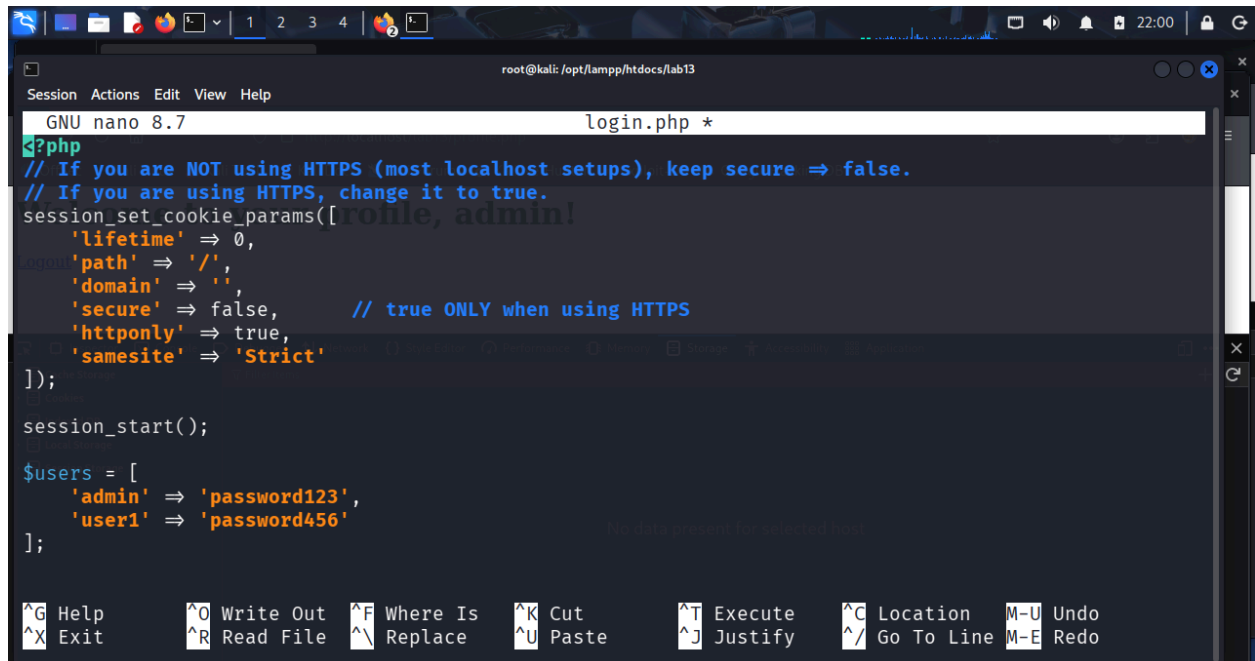
The following security improvements were added:

1. Secure Cookie Configuration

The session cookie was configured with:

- HttpOnly flag
- SameSite=Strict
- secure flag (depending on HTTP/HTTPS usage)

This reduces the risk of JavaScript-based cookie theft and cross-site attacks.



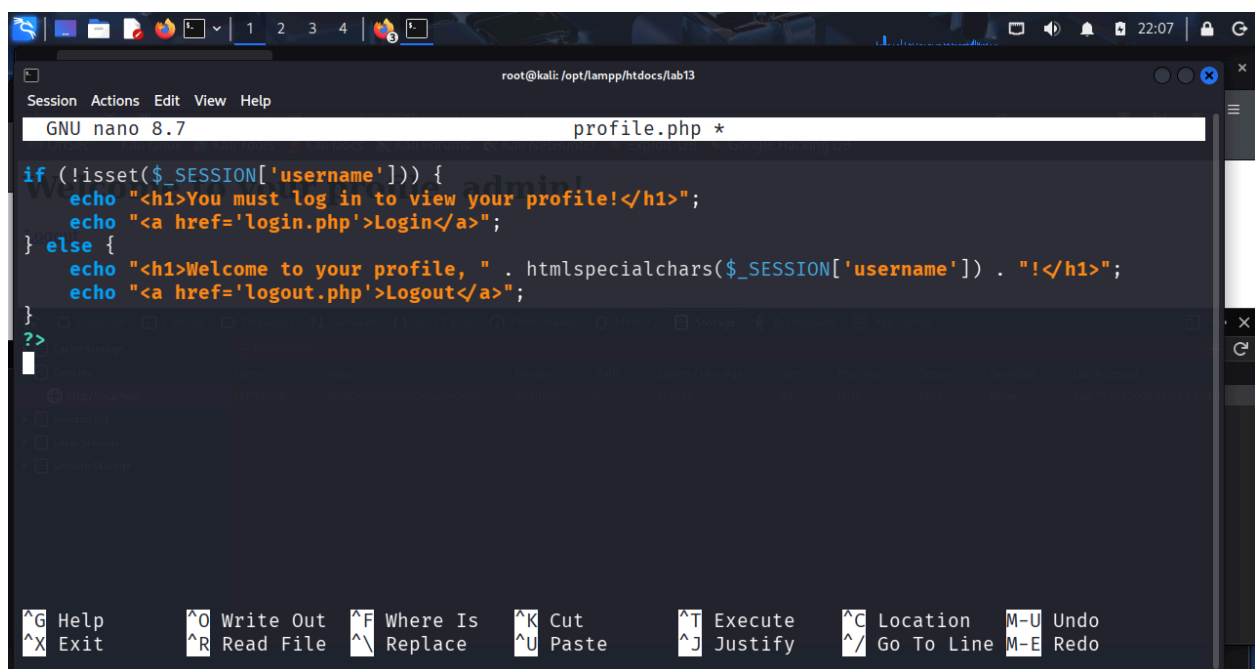
A terminal window showing the nano editor editing a file named `login.php`. The editor's title bar indicates the path `root@kali: /opt/lampp/htdocs/lab13`. The code in the file is as follows:

```
GNU nano 8.7 login.php *
<?php
// If you are NOT using HTTPS (most localhost setups), keep secure => false.
// If you are using HTTPS, change it to true.
session_set_cookie_params([profile, admin!
    'lifetime' => 0,
    'path' => '/',
    'domain' => '',
    'secure' => false,      // true ONLY when using HTTPS
    'httponly' => true,
    'samesite' => 'Strict'
]);

session_start();

$users = [
    'admin' => 'password123',
    'user1' => 'password456'
];
```

The bottom of the window shows a row of keyboard shortcuts: `^G Help`, `^X Exit`, `^O Write Out`, `^R Read File`, `^F Where Is`, `^N Replace`, `^K Cut`, `^U Paste`, `^T Execute`, `^J Justify`, `^C Location`, `^_ Go To Line`, `M-U Undo`, and `M-E Redo`.



A terminal window showing the nano editor editing a file named `profile.php`. The editor's title bar indicates the path `root@kali: /opt/lampp/htdocs/lab13`. The code in the file is as follows:

```
GNU nano 8.7 profile.php *

if (!isset($_SESSION['username'])) {
    echo "<h1>You must log in to view your profile!</h1>";
    echo "<a href='login.php'>Login</a>";
} else {
    echo "<h1>Welcome to your profile, " . htmlspecialchars($_SESSION['username']) . "!</h1>";
    echo "<a href='logout.php'>Logout</a>";
}
?>
```

The bottom of the window shows the same row of keyboard shortcuts as the previous screenshot.

2. Session ID Regeneration

After successful login:

```
session_regenerate_id(true);
```

This prevents session fixation and reuse of stolen session IDs.

3. Session Timeout

A 15-minute session timeout was implemented in profile.php:

```
$timeout_duration = 900;
```

If inactive beyond this period, the session is destroyed automatically.

Task 3: Testing Mitigation

1. Retesting Session Hijacking

The hijacking process was repeated after implementing security controls.

Result:

This attack should have failed if:

- The session ID changed after login.
- The previous session ID was no longer valid.

2. Testing Session Timeout

After remaining idle beyond the timeout duration:

Result:

The system should have displayed:

Your session has expired. Please log in again.

Conclusion

This lab demonstrated how session hijacking occurs through session ID theft. It also showed that implementing security best practices such as:

- Regenerating session IDs
- Using secure cookie flags
- Enforcing session timeouts
- Using HTTPS

significantly reduces the risk of session hijacking attacks.

The mitigation techniques successfully prevented unauthorized session reuse.